COM 307 Project 3

Random Forest for the Ionosphere Data

Muhammad Abdullah

Due: November 6th 2026

The data

The dataset has 351 radar returns, each described by 34 numbers: 17 pulses, with a real and an imaginary part per pulse (A1/A2 are pulse 1, A3/A4 pulse 2, and so on). The label is *good* (225 rows) or *bad* (126 rows), so always guessing "good" already gets 64.1%. Before building anything I looked at the data, and a few things shaped later decisions:

- **A2 is always 0**, so it can never be split on. **A1 is only 0 or 1**, all 38 rows with A1 = 0 are bad.
- **The file isn't shuffled.** Rows 1–200 are about 50/50, but rows 201–351 are 82% good. Taking the first rows as training data would give train and test sets with very different class mixes, so I always shuffled and stratified.
- **The real parts carry most of the signal**, especially at pulses 1–5. Most imaginary parts after pulse 9 barely differ between classes (`step1_pulse_profiles.png` in the submission).
- A27 is exactly 1.0 in 88 rows, 75 of them bad. Like A1 = 0, this looks like a sign of a poor-quality return.

What I built

Everything is written from scratch in Python and NumPy (`random_forest.py`). At each node the code draws m random features. For each one it sorts the rows, walks through them keeping a running count of good rows on the left, and scores every cut between two different values by the weighted Gini of the two sides. The best feature and threshold win. A row goes left if its value is $\leq$ the threshold. The same process then repeats on each side.

Decision points and what I chose:

- **Split measure:** Gini impurity, $1 - p_{\text{good}}^2 - p_{\text{bad}}^2$.
- **Candidate thresholds:** midpoints between consecutive distinct values, so every possible cut is tried once.
- **When to draw the m features:** fresh at every node, not once per tree. The assignment could be read either way. Drawing per node is how Breiman's random forest works, and it makes the trees more different from each other.
- **Which features can be drawn:** only features that still vary at that node. A2 never varies, and many features become constant deep in a tree. Drawing them would waste one of the m picks and could end a branch early for no reason.
- **Stopping:** a node becomes a leaf when it's pure, has fewer than 2 rows, or none of the drawn features improves Gini. There's no depth limit. A 50/50 leaf or vote predicts "good", the majority class.
- **Combining trees:** each tree is trained on a bootstrap sample (351 draws with replacement from the training rows), and the forest predicts by majority vote. The rows a tree never saw give a free

"out-of-bag" (OOB) estimate of accuracy. As an extra, I also find the single tree that agrees most often with the forest's vote.

- **Evaluation:** a fixed stratified 70/30 split (245 train, 106 test, seed 307). Every model is scored on the same 106 test rows, and the test rows are never used to choose anything.

To check the code, I compared my split search with a slow brute-force version on 200 random datasets, and the two always agreed. My best root split (A5 ≤ 0.2315) is the same one scikit-learn's one-level tree picks.

Choosing m

I ran 5-fold cross-validation, repeated three times, on the 245 training rows only, for ten values of m from 1 to 34. Each m was scored for a 100-tree forest and for single trees (Figure 1).

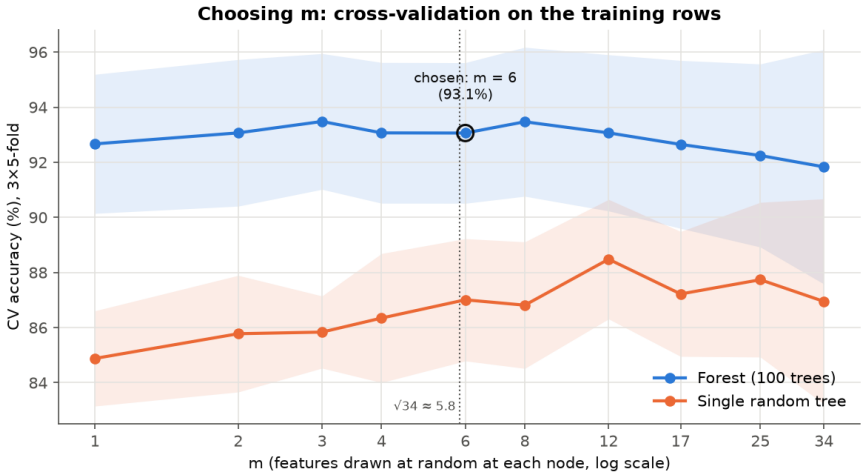


Figure 1: Cross-validation accuracy against m (shaded bands are ± 1 standard deviation across folds). The forest is flat from $m = 1$ to $m = 17$. Single trees improve as m grows.

The main thing I learned here is that the forest barely cares about m. Every value from 1 to 17 lands between 92.6% and 93.5%, while the fold-to-fold spread is about ± 2.5 points. Only $m = 25$ and $m = 34$ are clearly worse (92.2% and 91.8%) and less stable. My first version just took the highest score and picked $m = 2$. After I repeated the cross-validation it picked $m = 8$ instead, which convinced me that the "best" value was noise. So I used this rule: any m within 1 point of the best score counts as equally good, and among those I take the one closest to $\sqrt{34} \approx 5.8$, the usual default. **That gives $m = 6$.**

Single trees go the other way and do best around $m = 12$. A single tree just wants good splits, so seeing more features helps it. A forest also needs its trees to make *different* mistakes that cancel out in the vote; with $m = 34$ the trees pick nearly the same splits and the vote has little to average over.

Results

The first random tree grows to depth 10 with 25 leaves and fits the training data perfectly (100%), so it is clearly overfitting. Its accuracy also depends heavily on its random draws (79.2% to 91.5% across 20 seeds). The forest fixes both problems (Figure 2, left). It adds about 7 points of accuracy and cuts the seed-to-seed spread by about ten times. Accuracy stops improving after about 30 trees (Figure 2, right). The OOB estimate, computed from training rows only, ends up within a point of the test accuracy.

Model ($m = 6$, same 106 test rows)	Test accuracy	Bad rows correct
Always guess "good"	64.2%	0%
First random tree	87.7%	81.6%
20 single trees (different seeds)	87.0% ± 3.0%	–
Random forest, 100 trees	**93.4%**	**89.5%**
its out-of-bag estimate	93.5%	–
20 forests (different seeds)	94.2% ± 0.3%	–
scikit-learn forest, same settings (check only)	94.3% ± 0.2%	–

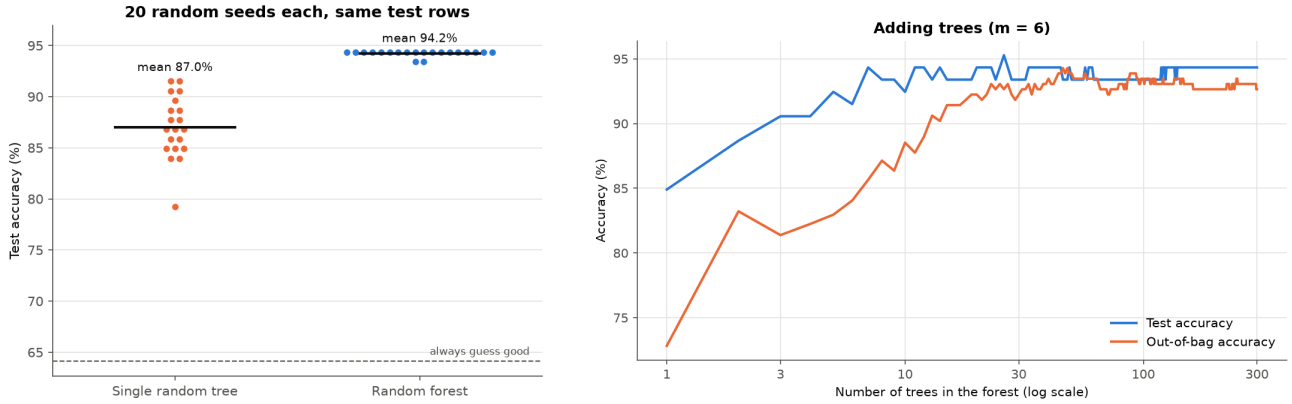


Figure 2: Left: test accuracy for 20 single trees and 20 forests. Right: test and out-of-bag accuracy as trees are added to one forest.

Which features matter. Four real-part features (A5, A7, A27, A3) account for about half of the forest's total Gini reduction. These were also the best single splits at the root. More surprisingly, imaginary parts (A8, A28, A14, A18, A16) take ranks 5–9, even though on their own they separate the classes poorly. They seem to be useful for cleaning up the cases left over after the first big splits. Overall, real parts account for 65% of the importance and imaginary parts for 35%.

The original 1989 split. Training on rows 1–200 and testing on rows 201–351, as the original paper did, the forest gets 95.4% and catches 25 of the 27 bad rows. That is in the same range as the published results (nearest neighbor 92.1%, C4 94.0%, backprop ∼96%); with 151 test rows, one row is worth about 0.7 points, so these gaps are small. This test set is 82% good, though, so its accuracy isn't directly comparable to my stratified split.

Limitations. With 106 test rows, one row is worth 0.9 points, so gaps of a point or two between models don't mean much. "Bad" returns are consistently harder to classify than good ones, as the original paper also found.

Use of generative AI

I used Claude to help me with this project. It helped me plan the steps and do the EDA. I wrote the code myself (data exploration, tree and forest, tests, and the experiment), but used claude to format the print/log statements nicely and make plots. It also helped me convert the google doc version of the write-up to latex code.